AULA 10 · EXERCÍCIO FINAL · 5 HORAS

Assistente de Gestão de Pessoas

Férias, abonos e diárias: do texto livre do servidor ao registro auditado, passando por agentes, MCP, grafo, chefia e segurança.

Aulas 1 a 9 | pasta aula10/ | PASSO_A_PASSO.md | python conferir.py

O PROBLEMA

“Quero tirar férias de 12 a 26/04 e vender 10 dias.” O que o sistema precisa fazer com esse texto?

1 · proteger
CPF, saúde, fora do escopo, manipulação

2 · entender
tipo, matrícula, datas, dias vendidos, destino

3 · conferir
cadastro, escala da equipe e feriados

4 · analisar
especialistas: Normas, Escala, Financeiro

5 · redigir
o despacho, e validá-lo antes de seguir

6 · pedir aprovação
à chefia, que pode responder dias depois

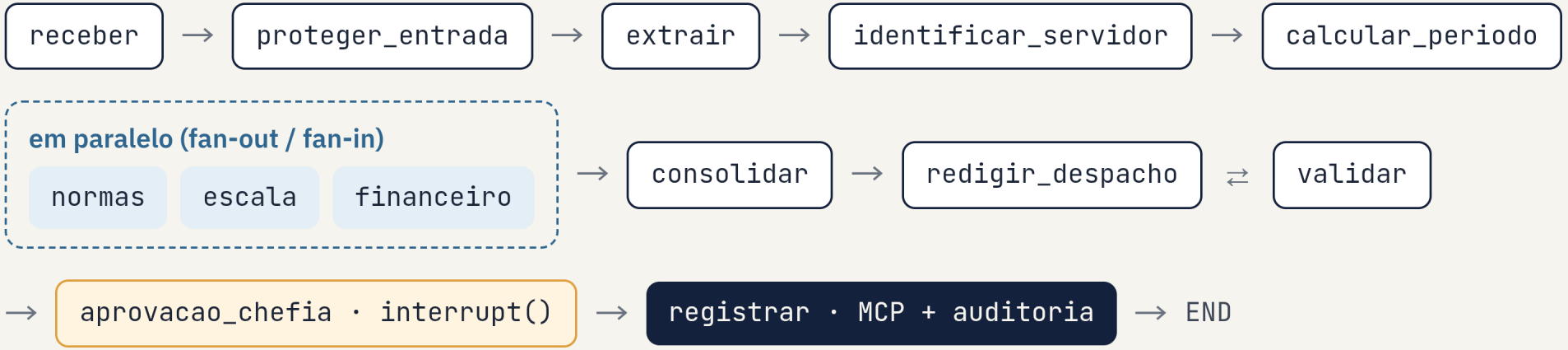
7 · registrar
sem duplicar, com auditoria

8 · deixar rastro
tempo e tokens de cada passo

O domínio é novo: os exemplos das aulas falam de ocorrências. Você vai **adaptar**, não copiar.

ONDE VOCÊ VAI CHEGAR

O sistema inteiro é um grafo LangGraph



bloqueado na entrada → recusar
despacho inválido → redige de novo (máx. 3)

incompleto ou servidor não encontrado → pedir_dados
chefia rejeitou → redige de novo (máx. 2)

Diagrama completo no ENUNCIADO.md · gere o seu com: python etapa07_grafo.py --mermaid

COMO TRABALHAR

Troque cada raise pelo seu código

Cada lacuna é um comentário com a referência da aula e um **raise**:

```
# ETAPA 1.2 -- Agent(name="AtendenteRH", ...)
# Referência: aula1/segundo_agente.py
raise NotImplementedError("ETAPA 1.2: ...")
```

Trechos **(PRONTO)** não são o foco: use sem medo. Cada arquivo abre com um bloco **REFERÊNCIAS NAS AULAS**.

Ao fim de cada etapa: python conferir.py N

- `[OK]` pronto, siga em frente
- `[PENDENTE]` ainda tem o raise
- `[FALHOU]` rodou, resultado errado: leia o esperado
- `[ERRO]` exceção: a mensagem diz a linha

Não trave. Passou de 1,5× o tempo sugerido? Deixe um # TODO explicando o que falta e siga.

O conferir.py usa respostas fixas no lugar do LLM nas Etapas 7 a 10: não gasta tokens. Use --llm para incluir o modelo.

Cronograma: 5 horas, dois marcos

#	Etapa	min	relógio
0	Preparação e leitura da norma	10	0:00
1	Primeiro agente com tool	15	0:10
2	Extração estruturada	15	0:25
3	Guardrails de entrada	15	0:40
4	APIs com resiliência (pronto)	10	0:55
5	MCP Server do RH	30	1:05
–	Intervalo	10	1:35

#	Etapa	min	relógio
6	Especialistas e fluxo linear ⇒ Marco mínimo	30	1:45
7	O grafo (LangGraph)	55	2:15
8	Persistência e aprovação da chefia	25	3:10
9	Segurança ⇒ Marco completo	25	3:35
10	Observabilidade	10	4:00
11	Demonstração e entrega	20	4:10
–	Folga ou opcionais 🆕 A, B, C	30	4:30

Marco mínimo · 2:15
o fluxo linear processa um pedido de ponta a ponta

Marco completo · 4:00
grafo + chefia + segurança funcionando

Os opcionais 🆕 só depois do Marco completo. Uma etapa incompleta que roda vale mais que um sistema perfeito parado na Etapa 4.

ETAPA 0 · 10 MIN · AULA 1

Preparação: ambiente e a norma

1. Confira o ambiente e os dados:

```
python verificar_ambiente.py --sem-llm
python dados_rh.py
```

Esperado: 9 servidores; 3 afastamentos; 13 feriados locais; 10 regras

2. Leia dados/normas.md (N1 a N10) e preencha no ENTREGA.md a tabela **quem verifica cada regra**.

Dica para a tabela

É uma **conta**? (saldo, datas, limite da escala) → **função Python**

Exige **interpretar** texto? → **agente**

É a **decisão de alguém**? (N8, a chefia) → **humano**

Guarde a tabela: na Etapa 6.0 você compara com o etapa06_regras.py.

Conferir: python conferir.py 0

ETAPA 1 · 15 MIN · ETAPA01_AGENTE.PY

Primeiro agente com tool

Lacuna	O que fazer	Onde olhar
1.1	Tool consultar_servidor: print, docstring com Args: e o return de servidor_publico(...)	exercicios_resolvidos/aula4/ ex10_function_tool.py
1.2	criar_atendente(): Agent "AtendenteRH" com a tool	aula1/segundo_agente.py
1.3	main(): as duas perguntas com Runner.run_sync	aula8/exemplos/02_agente_tool/main.py

```
> python etapa01_agente.py
[tool] consultar_servidor 1002
A matrícula 1002 (Bruno Castro) tem
30 dias de férias disponíveis.
```

Confira

[tool] aparece **uma vez** (só na 1ª pergunta). Duas? Melhore as instruções. Nenhuma? Melhore a docstring.

O **salário** nunca aparece na resposta.

Conferir: python conferir.py 1 · Aulas 1 e 4

ETAPA 2 · 15 MIN · ETAPA02_EXTRACAO.PY

Extração estruturada do pedido

Lacuna	O que fazer	Onde olhar
2.1	Campos do Pedido com Field(description=...)	aula2/agente_output2.py
2.2	criar_extrator() com output_type=Pedido	aula2/agente_output.py
2.3	extrair(): Runner.run_sync(...).final_output	aula2/agente_output.py

Dica: o modelo inventa datas

O C5 diz “agosto de 2027”: isso **não** é data exata. Diga na description o que fazer quando falta a informação:

```
data_inicio: str | None = Field(None,
  description="AAAA-MM-DD; null se o
  servidor não disse o dia exato")
```

Valores corretos

C2 férias · 1002 · 2027-04-12 a 2027-04-26 · vende 10

C5 férias · matrícula, datas = **None**

C11 diária · 1003 · Buenos Aires · exterior

A data de hoje já vai nas instruções (texto_hoje): “amanhã” vira uma data real.

Conferir: python conferir.py 2 · Aula 2 · opcional 🆕 A também está neste arquivo (deixe para o fim)

ETAPA 3 · 15 MIN · ETAPA03_GUARDRAILS.PY

Guardrails de entrada

Lacuna	O que fazer	Onde olhar
pronto	RE_CPF, RE_CID e encontrar_dado_sensivel(): leia e use	exercicios_resolvidos/ aula4/ex11_guardrails.py
3.2	criar_classificador() com output_type=AvaliacaoEscopo	aula2/agente_guardrail.py
3.3	nota_escopo(texto) → (nota, motivo)	aula2/agente_guardrail.py
3.4	Os dois @input_guardrail (motivo em output_info)	aula2/agente_guardrail.py
3.5	main(): trate InputGuardrailTripwireTriggered	aula8/.../04_prompt_injection

```
> python etapa03_guardrails.py    # as notas variam; quem passa, não
C1_abono_simples    nota=0.05  PASSOU (tipo=abono)
C3_dado_de_saude    BLOQUEADO (cpf): Por segurança, não envie CPF...
C4_fora_do_escopo   nota=0.95  BLOQUEADO (fora_do_escopo)
C7_autoaprovacao    nota=0.90  BLOQUEADO (fora_do_escopo)
C10_zona_cinzenta   nota=0.50  PASSOU        ← anote esta nota (🆕 C)
```

Conferir: python conferir.py 3 · Aulas 2, 4 e 8 · próximo slide: dicas

DICAS · ETAPA 3

Guardrail async e classificador com exemplos

1. Dentro do guardrail, use await

nota_escopo() usa run_sync e não funciona dentro de código async. Chame o classificador assim:

```
@input_guardrail
async def barrar_fora_do_escopo(ctx, agent, entrada):
  r = await Runner.run(criar_classificador(),
        entrada, context=ctx.context)
  nota = r.final_output.nota
  return GuardrailFunctionOutput(
    output_info={"motivo": ...},
    tripwire_triggered=...)
```

2. Exemplos nas instruções

Nota **baixa**: pedidos de férias, abono, diária. Nota **alta**: outro assunto **e tentativa de mudar as regras** (“ignore as regras”, “eu mesmo aprovo”). Sem isso, o C7 passa.

3. Calibre pelo C10

A nota do C10 deve ficar entre ~0,3 e ~0,7. Perto de 0 ou de 1? Ajuste os exemplos.

4. Recupere o motivo no except

```
e.guardrail_result.output.output_info["motivo"]
```

A regex não pode casar com datas: “12/04/2027” não é CPF nem CID. Nos 12 casos, só o C3 dispara a regex.

ETAPA 4 · 10 MIN · ETAPA04_FERRAMENTAS.PY · PRONTO

APIs com resiliência: leia, rode e explique

Função	Compare com
get_json_com_retry()	aula3/agente_retry2.py
buscar_feriados()	aula6/exemplos/11_grafo_real
cotacao_usd_brl()	aula3/agente_retry4.py
gerar_protocolo()	aula3/agente_retry3.py
rodar_com_limites()	aula3/agente_parada3.py

```
> $env:FERIADOS_OFFLINE = "1"
> python etapa04_ferramentas.py
[retry] tentativa 1 falhou ... (3x)
feriados 2027: 13 (fonte: arquivo_local)
dias úteis 12/04 a 26/04/2027: 10
protocolo C1=ae788d33 C8=ae788d33 iguais=True
```

Online: sem as linhas [retry], fonte: brasilapi e uma cotação de verdade.

Responda no ENTREGA.md

Por que a espera **dobra** a cada tentativa? · Por que o protocolo é um **hash** dos campos e não um número sequencial? · O que aconteceria sem o **cache** de feriados?

Conferir: python conferir.py 4 · Aulas 3 e 6 · o protocolo ae788d33 vai aparecer em todas as saídas daqui para frente

ETAPA 5 · 30 MIN · ETAPA05_MCP_RH.PY E ETAPA05_CLIENTE_MCP.PY

O MCP Server do RH

Lacuna	O que fazer	Onde olhar
5.1	5 tools de leitura: @mcp.tool(), docstring e o return da consulta pronta	aula6/desafio2/mcp_unidades.py
5.2	registrar_decisao idempotente (só gravar; a segurança é a 9.2)	aula3/agente_retry3.py
5.3	perguntar_ao_atendente() com MCPServerStdio	aula5/exemplos/04_agente_mcp

TESTE 1 · o server sozinho

```
npx @modelcontextprotocol/ inspector
python etapa05_mcp_rh.py
```

Connect → Tools → List: **6 tools**. Chame consultar_servidor com 1002.

TESTE 2 · sem agente

```
python etapa05_cliente_mcp.py
```

1ª gravação: sucesso. 2ª: **duplicado**. O 1002 vem **sem salário**; Cartório: limite **1**.

TESTE 3 · com agente

```
python etapa05_cliente_mcp.py
--agente
```

Responde que a 1006 está de férias de 05/07 a 19/07/2027.

Conferir: python conferir.py 5 · Aula 5 · o Inspector precisa de Node.js · próximo slide: dicas

DICAS · ETAPA 5

Um dict por tool, e o protocolo manda

1. Toda tool devolve UM dict

Uma tool que devolve **lista** manda cada item separado: o cliente não distingue "lista com 1 regra" de "1 dict". Embrulhe:

```
return {"tema": tema, "regras": normas(tema)}
```

2. A docstring é o schema

É o que o LLM lê para decidir. Diga o formato das datas (AAAA-MM-DD), os temas que existem e o que volta.

3. Idempotência = olhar antes de gravar

```
registros = _ler_registros()
if protocolo in registros:
  return {"sucesso": True, "duplicado": True, ...}
registros[protocolo] = {...}  # e grave
```

Error executing tool consultar_xxx

O server **esconde** o detalhe do erro. Chame a mesma tool no **MCP Inspector**: lá aparece o traceback.

asyncio.run() cannot be called from a running event loop

Você usou chamar_mcp (síncrono) dentro de um **async def**. Lá dentro, use **await chamar_mcp_async(...)**.

5.3 · o agente com o server

async with MCPServerStdio(params=...) **as** server, e o Agent recebe **mcp_servers=[server]** (não tools=).

O salário só sai por consultar_remuneracao, marcada [RESTRITA]: na Etapa 9 só o Financeiro poderá usá-la.

ETAPA 6 · 30 MIN · ETAPA06_AGENTES.PY E ETAPA06_LINEAR.PY

Especialistas e o fluxo linear

Lacuna	O que fazer	Onde olhar
6.0	Pronto: rode etapa06_regras.py e compare com a sua tabela N1–N10	aula6/exemplos/11_grafo_real
6.1	Normas, Escala, Financeiro e Redator, cada um só com as suas tools	aula7/exemplos/ 02_agentes_especializados
6.2	Coordenador com os 3 especialistas como as_tool	aula4/ex03_agente_as_tool.py
6.3	analisar(): o Coordenador com rodar_com_limites_sync	aula4/ex03_agente_as_tool.py
6.4	processar(): os 9 passos, imprimindo cada um	aula6/exemplos/01_fluxo_linear

Normas
consultar_normas · explica as regras

Escala
consultar_escala · N5 e sugere datas

Financeiro
diárias, remuneração, dólar · único que vê o salário

Redator
sem tools · escreve o despacho com os fatos

Conferir: python conferir.py 6 · Aulas 2, 4 e 7 · rode: python etapa06_linear.py (C1, C2, C6 e C11) · próximo slide: dicas

DICAS · ETAPA 6

Especialistas viram tools do Coordenador

1. O padrão do as_tool

```
criar_normas().as_tool(
  tool_name="parecer_normas",
  tool_description="Regras aplicáveis...",
  hooks=hooks,
  failure_error_function=None)
```

hooks=hooks: o hook enxerga as tools **dentro** de cada especialista. **failure_error_function=None**: um erro lá dentro interrompe, em vez de virar um texto que o Coordenador ignora.

Use ModelSettings(temperature=0) nos especialistas para respostas mais estáveis (aula7/agentes.py).

2. Confira as contas à mão

Caso	Valor esperado
C2	10 dias × R$ 9.500 / 30 = R$ 3.166,67
C11	3 pernoites + meia = 3,5 × USD 320 = USD 1.120
C6	indeferido (N5); sugere datas após 19/07/2027

3. Protocolos que não mudam

`C1 ae788d33 · C2 b7b6f485`
`C6 6f1b47d7 · C11 ec67dba1`

Diferentes? O problema está na **extração**, não no protocolo.

Depois da Etapa 9, o C2 e o C11 deste fluxo passam a ser negados pelo server (falta a chefia). É o comportamento certo.

2:15 · MARCO MÍNIMO

Um pedido, de ponta a ponta.

Guardrail → extração → MCP → feriados → regras → especialistas → despacho → registro.
Funciona, mas é uma linha reta: não decide, não volta, não espera ninguém.
A Etapa 7 resolve isso.

```
python conferir.py 0 1 2 3 4 5 6
```

ETAPA 7 · 55 MIN · ETAPA07_GRAFO.PY · A MAIS LONGA

O grafo: faça nesta ordem

Ordem	O que fazer	Onde olhar
1 · 7.1	reducer em violacoes	aula7/exemplos/08_paralelismo
2 · 7.5 arestas	construir(): o diagrama, sem chefia e sem triagem	aula6/exemplos/06 e 07
3 · 7.2	entrada, extração, servidor, período e roteadores	aula6/desafio2/main.py
4 · 7.3	os três especialistas e o consolidar	aula7/exemplos/08_paralelismo
5 · 7.4	redigir_despacho, validar e o ciclo	aula6/exemplos/08_fluxo_ciclico
6 · 7.5 registrar	registrar() chama registrar_decisao no MCP	aula6/desafio2/main.py

Por que essa ordem?

Com as arestas prontas cedo, o grafo compila e você testa **caso a caso** à medida que preenche os nós.

Até a Etapa 8

rotear_validacao manda “chefia” também para “registrar”. Deixe USAR_VOTACAO e USAR_TRIAGEM_HUMANA em False.

DICAS · ETAPA 7 (1 DE 2)

Nó devolve o que mudou; fan-in por lista

1. Um nó devolve um dict parcial

Só as chaves que ele mudou, sempre com o próprio nome no caminho (o reducer de caminho acrescenta):

```
def identificar_servidor(estado):
  s = chamar_mcp("consultar_servidor",
    {"matricula": estado["pedido"]["matricula"]})
  return {"servidor": s,
          "caminho": ["identificar_servidor"]}
```

2. O roteador só lê e devolve um rótulo

rotear_servidor: "nao_encontrado" se veio {"erro": ...}, senão "ok". O rótulo é a **chave** do dicionário em add_conditional_edges.

3. Fan-out e fan-in

```
for e in ("normas", "escala", "financeiro"):
  c.add_edge("calcular_periodo", e)
c.add_edge(["normas", "escala",
            "financeiro"], "consolidar")
```

A **lista** é a junção: consolidar só roda depois que os três terminaram, e roda uma vez.

4. Cada ramo escreve no SEU campo

no_normas → parecer_normas; no_escala → parecer_escala. Campo escrito por dois ramos no mesmo passo precisa de reducer:

```
violacoes: Annotated[list[str], operator.add]
```

Experimento da 7.1: faça no_escala devolver {"violacoes": ["teste"]} com e sem o reducer e compare.

DICAS · ETAPA 7 (2 DE 2)

Um ciclo precisa de um contador

redigir_despacho ⇄ validar

validar usa problemas_do_despacho() (pronta) e **soma 1** em tentativas. O roteador decide com essa conta:

```
def rotear_validacao(estado):
  if estado["erro_validacao"] and \
     estado["tentativas"] < MAX_TENTATIVAS:
    return "redigir"
  return ...  # "chefia" ou "registrar"
```

No redigir_despacho, passe ao Redator o **erro_validacao** (e, na Etapa 8, o **feedback_chefia**): é assim que ele corrige.

No registrar: sem chefia, aprovador = "sistema"; o token vem de os.getenv("TOKEN_REGISTRO").

Se aparecer...

GraphRecursionError

O ciclo não para: tentativas não aumenta, ou o roteador não lê o limite.

InvalidUpdateError: At key 'violacoes'

Dois ramos paralelos escreveram no mesmo campo sem reducer (7.1).

Erro de rótulo no roteador

O rótulo devolvido não está no dicionário do add_conditional_edges.

Desenhe o seu grafo: python etapa07_grafo.py --mermaid e cole em mermaid.live

ETAPA 7 · SAÍDA ESPERADA

Compare o seu caminho com este

```
> python etapa07_grafo.py C1_abono_simples

[C1_abono_simples] caminho: receber → proteger_entrada → extrair → identificar_servidor → calcular_periodo → escala
→ financeiro → normas → consolidar → redigir_despacho → validar → registrar

[C1_abono_simples] resultado: registrado  protocolo: ae788d33

> python etapa07_grafo.py C6_conflito_de_escala   # mesmo caminho, despacho de INDEFERIMENTO (N5), 6f1b47d7
```

- ✓ Os três ramos em azul podem vir em **qualquer ordem**, mas todos antes de consolidar, que aparece **uma vez**.
- ✓ **redigir_despacho -> validar** repetido até 3 vezes é normal. Olhe o erro_validacao.
- ✓ Rodou o C1 de novo e deu **duplicado**? Certo: o protocolo já existe. Apague saidas/ para recomeçar.
- ✓ Os caminhos **não dependem do LLM**, desde que a extração esteja certa.

Conferir: python conferir.py 7

ETAPA 8 · 25 MIN · ETAPA07_GRAFO.PY

A chefia pode responder dias depois

Lacuna	O que fazer	Onde olhar
8.1	Pronto: o main() já compila com SqliteSaver. Leia.	aula7/exemplos/04_persistencia
8.2	aprovacao_chefia() com interrupt()	aula7/exemplos/07_aprovacao_revisao
8.3	rotear_chefia() e as arestas validar → chefia → redigir/registrar	aula7/exemplos/07_aprovacao_revisao
8.4	retomar() com Command(resume=...)	aula7/exemplos/06_human_in_the_loop

```
> python etapa07_grafo.py C2_ferias_com_venda
[C2_ferias_com_venda] PAUSADO em ('aprovacao_chefia',): Aprova o despacho?   ← o processo TERMINA aqui
> python etapa07_grafo.py --retomar C2_ferias_com_venda "nao:confirme o valor da venda" 1010
... → aprovacao_chefia → redigir_despacho → validar   PAUSADO de novo
> python etapa07_grafo.py --retomar C2_ferias_com_venda sim 1010
... → aprovacao_chefia → registrar   registrado b7b6f485
```

Conferir: python conferir.py 8 · Aulas 3 e 7 · veja quem está pausado: python etapa07_grafo.py --pendentes · C11 aprova com 1011

DICAS · ETAPA 8

interrupt() pausa; ao retomar, o nó recomeça

1. O que o interrupt devolve

É o valor que o retomar() passa no Command(resume=...). Combine o formato dos dois lados:

```
resposta = interrupt({"pergunta": "Aprova?",
  "chefia": ..., "resumo": ...})
# resposta = {"aprovado": bool,
#             "aprovador": str, "motivo": str}
```

2. Nada de efeito antes do interrupt

Na retomada, o nó roda **desde o começo**. Teste: um print("ANTES") no início aparece duas vezes. Gravar algo ali duplicaria.

3. Onde parou?

```
proximo = app.get_state(config_de(t)).next
if not proximo:
  print(f"[{t}] nada pausado"); return
saida = app.invoke(Command(resume=valor),
                   config_de(t))
```

4. O ciclo da chefia

Rejeitou → soma 1 em rejeicoes e guarda o motivo em feedback_chefia. Volta a redigir até **MAX_REJEICOES**; depois registra como **indeferido_pela_chefia**.

O que faz a pausa sobreviver ao fim do processo é o checkpointer (SqliteSaver em saidas/checkpoints.db), não a memória.

ETAPA 9 · 25 MIN · ETAPA05_MCP_RH.PY E ETAPA07_GRAFO.PY

Segurança: quem nega é o server

Lacuna	O que fazer	Onde olhar
9.1	Só conferir: nenhum agente tem registrar_decisao; só o Financeiro tem consultar_remuneracao	aula8/.../06_excessive_agency
9.2	Em registrar_decisao: token, decisão válida, autoaprovação e chefia certa (N8)	aula5/exemplos/10_seguranca/ server_seguro.py
9.3	_auditar(): uma linha JSON por tentativa, ALLOW ou DENY	aula5/.../server_seguro.py
9.4	No registrar do grafo: não grava despacho com CPF/CID	aula8/.../05_guardrail_saida

```
> ... --retomar C2_ferias_com_venda sim 1002
negado_pelo_server: autoaprovação (N8)
> ... --retomar C2_ferias_com_venda sim 1011
negado_pelo_server: aprovador não é a chefia 1010 (N8)
```

```
> python etapa07_grafo.py --sem-guardrail
C7_autoaprovacao
"eu aprovo" não aprova nada: as regras indeferem (N4,
menos de 30 dias de antecedência).
```

Conferir: python conferir.py 9 · Aulas 5 e 8 · rode o C2 de novo entre um teste e outro (o anterior terminou)

DICAS · ETAPA 9

Negue primeiro, grave depois, audite sempre

1. Uma função para negar

```
def negar(motivo):
  _auditar(protocolo, "DENY", motivo, parametros)
  return {"sucesso": False, "erro": motivo}
```

2. A ordem das conferências

1. token diferente de TOKEN_REGISTRO (ou vazio)
2. decisao fora de DECISOES_VALIDAS
3. aprovador == matricula (autoaprovação)
4. deferido + tipo com chefia + aprovador ≠ chefia do **cadastro**
5. só então: idempotência, gravar e auditar ALLOW

Por que no server?

O prompt do agente pode ser manipulado; o código do server, não. A chefia certa vem do **cadastro**, nunca do que o pedido diz.

A auditoria

Uma linha JSON por tentativa, em modo de acréscimo (“a”). **Nunca** grave o token nem o despacho: só quando, quem, protocolo, ALLOW/DENY, motivo e parâmetros.

9.4 · guardrail de saída

No registrar, antes do MCP: encontrar_dado_sensivel(despacho) → não grava, resultado “bloqueado_na_saida”.

Token sempre negado? Confira TOKEN_REGISTRO no .env e salve o arquivo: o server lê o .env sozinho.

4:00 · MARCO COMPLETO

Decide, volta, espera e se protege.

O grafo escolhe o caminho, refaz o despacho, pausa para a chefia, sobrevive ao fim do processo e o server nega o que não pode.

```
python conferir.py
```

ETAPA 10 · 10 MIN · ETAPA10_GOVERNANCA.PY E ETAPA10_OBSERVABILIDADE.PY

Observabilidade: tempo e tokens por nó

Lacuna	O que fazer	Onde olhar
10.1	on_tool_end e on_agent_end acrescentam a self.eventos; passe hooks=Governanca() nos nós	aula3/agente_hook2.py
10.2	medido(): uma linha por nó em saidas/metricas.jsonl	aula8/.../03_observabilidade

Dica: medido() embrulha o nó

```
@functools.wraps(no)
def medido_no(estado):
  t0 = time.perf_counter()
  r = no(estado)
  ... grave a linha JSON ...
  return r  # o MESMO dict
```

```
> python etapa10_observabilidade.py
  C2_ferias_com_venda
"segundos_total": 14.7,
"mais_lento": ["financeiro", 4.2],
"tokens_total": 9830
```

Os números variam; a forma, não. Por que identificar_servidor leva ~1 s sem LLM? Cada chamar_mcp sobe o server de novo: o que mudaria em produção?

Conferir: python conferir.py 10 · Aulas 3, 5 e 8 · resumo(thread_id) já vem pronto: ele lê o JSONL que o seu medido() grava

ETAPA 11 · 20 MIN · ENTREGA.MD

Demonstração: os 12 casos do zero

Apague saidas/ e rode:

```
python etapa07_grafo.py --todos
python etapa07_grafo.py --retomar
  C2_ferias_com_venda sim 1010
python etapa07_grafo.py --retomar
  C11_diaria_exterior sim 1011
python etapa07_grafo.py --mermaid
```

Preencha a tabela do ENTREGA.md e compare. C9 e C10 dependem do LLM: registre o que aconteceu.

Caso	Resultado	Protocolo
C1	registrado (deferido)	ae788d33
C2	chefia 1010 → registrado	b7b6f485
C3, C4, C7	recusado na entrada	
C5, C9	pediu dados	
C6	registrado (indeferido, N5)	6f1b47d7
C8	duplicado	ae788d33
C10	pediu dados (ou recusado)	
C11	chefia 1011 → registrado	ec67dba1
C12	registrado (indeferido, N3)	7c458e62

Conferir: python conferir.py 11 · referência completa dos caminhos no PASSO_A_PASSO.md

OPCIONAIS 🆕 · BÔNUS · SÓ DEPOIS DO MARCO COMPLETO

Um conceito conhecido, num contexto novo

🆕 A

Votação em paralelo

Aula 7: especialistas **diferentes** ao mesmo tempo, para ganhar tempo.

Aqui: a **mesma** extração 3 vezes e votar, para ganhar confiabilidade.

`etapa02_extracao.py`

🆕 B

Hooks que barram

Aula 3: RunHooks para **observar** turnos e tools.

Aqui: para **controlar**: barrar tool que não é do agente ou orçamento estourado.

`etapa10_governanca.py`

🆕 C

Humano na entrada

Aula 7: interrupt() para **aprovar a saída**.

Aqui: o guardrail ficou em dúvida; um atendente decide se o pedido **entra**.

`etapa07_grafo.py`

Vêm desligados (USAR_VOTACAO e USAR_TRIAGEM_HUMANA = False). Cada um vale até +5 pontos. Não há exemplo pronto: o mecanismo você conhece.

Conferir: python conferir.py --opcionais (ou só um: python conferir.py 🆕A)

DICAS · OPCIONAIS 🆕

O mecanismo você já tem; mude o propósito

🆕 A · votação

3 instruções **diferentes** em INSTRUCOES_VOTANTES, um extrator para cada, e rode juntos:

```
res = await asyncio.gather(
  *(Runner.run(e, texto)
    for e in extratores))
```

votar() já vem pronto. Paralelo ≈ tempo de **uma** chamada; sequencial ≈ **três**. O C9 deve sair com divergencia=True.

🆕 B · hooks que barram

on_tool_start roda **antes** da tool: levantar ali impede a execução.

```
if tool.name not in \
   PERMISSOES.get(agent.name, set()):
  raise LimiteExcedido(...)
```

Nas PERMISSOES do Coordenador use os **tool_name das as_tool** (parecer_normas...). A linha "EXECUTOU" não pode aparecer no teste (a).

🆕 C · humano na entrada

Troque o limiar único por **duas faixas**:

```
nota < LIMIAR_OK      → "ok"
nota ⩾ LIMIAR_BLOQUEIO → "bloqueado"
no meio               → "duvida"
```

"duvida" vai para triagem_humana, que usa interrupt() como a chefia. O C10 não pausa? Recalibre com a nota anotada na Etapa 3.

Ligue cada um só depois de fazê-lo: USAR_VOTACAO, USAR_TRIAGEM_HUMANA, python etapa06_linear.py --governanca

QUANDO TRAVAR

Problemas comuns

Sintoma	O que fazer
NotImplementedError: ETAPA n.m	Implemente a lacuna citada (às vezes de outro arquivo)
NotImplementedError: 🆕 ...	Você ligou um opcional sem fazê-lo: volte a chave para False
PROVEDOR=openai exige OPENAI_API_KEY	Preencha o .env e rode de dentro de aula10/
HTTP Error 403 na BrasilAPI	Faltou o User-Agent na requisição
Error executing tool consultar_xxx	Chame a tool no MCP Inspector para ver o traceback
asyncio.run() ... running event loop	Dentro de async def, use await chamar_mcp_async(...)
InvalidUpdateError: At key 'violacoes'	Reducer da 7.1: Annotated[list[str], operator.add]
GraphRecursionError	O ciclo não para: confira tentativas e rejeicoes
negado_pelo_server: token inválido	Confira TOKEN_REGISTRO no .env
duplicado onde esperava registrado	O protocolo já existe: apague saidas/

Lista completa no fim do PASSO_A_PASSO.md

Implemente.
Rode. Confira. Siga.

O QUÊ	COMO	ESTÁ CERTO?	ENTREGA
ENUNCIADO.md	PASSO_A_PASSO.md	python conferir.py N	ENTREGA.md

Uma etapa incompleta que roda vale mais que um sistema perfeito parado na Etapa 4.